

Data and Systems

This document is the contract. Everything here is what the department can give you on day one. Anything not listed here, you do not have — though you are free to argue that you need it, and to say how you would get it.

All figures are for **one subdivision** of the city. You are not being asked to handle the whole state.

1. Scale

Thing	Count
66/11 kV substations	4
11 kV feeders	31
Distribution transformers	412
LT poles	38,400
Poles with a telemetry device fitted	34,900 (≈91%)
Households served	≈ 210,000
Outage events per day, typical	12–18
Outage events per day, monsoon peak	up to 120

Poles per DT ranges from 9 to about 240, median around 70. LT lines run up to about 1.4 km from the transformer, with one to five branches off the main run.

Telemetry volume: heartbeats every 15 minutes from 34,900 devices is roughly **39 messages/second** steady state, with bursts of a few thousand messages in the seconds after a large outage.

2. Telemetry from the pole devices

Devices push to an HTTPS endpoint you expose. (In production they publish over NB-IoT to an MQTT broker; for this exercise, assume you can define the ingest interface, and say in your architecture doc how you would adapt it.)

Payload

```
{
  "device_id": "KSPDB-SD07-D0112-4431",
  "pole_id": "P-024431",
  "event": "power_lost",
  "energized": false,
  "ts": "2026-07-29T02:14:07.412Z",
  "seq": 88213,
  "battery_mv": 3480,
  "rssi": -91,
  "fw": "1.4.2"
}
```

Field

Notes

Field	Notes
<code>device_id</code>	Stable per physical device. Devices get swapped; the same pole can change <code>device_id</code> over time.
<code>pole_id</code>	Foreign key into the pole registry. Trust this over <code>device_id</code> for location.
<code>event</code>	One of <code>heartbeat</code> , <code>power_lost</code> , <code>power_restored</code> , <code>boot</code> .
<code>energized</code>	Current state as the device sees it.
<code>ts</code>	Device clock. Skew up to ±90 seconds . Do not assume monotonicity across devices.
<code>seq</code>	Monotonic per device, resets to 0 on <code>boot</code> . Your one reliable tool for ordering and de-duplication within a device.
<code>battery_mv</code>	Reserve capacitor voltage. Below ~3200 the device may fail to send its dying message.
<code>rss_i</code>	Radio signal strength. Useful for telling "this device is in a bad coverage spot" from "this device is dead."
<code>fw</code>	~8% of the fleet is on firmware 1.2.x, which does not send <code>power_lost</code> at all — it simply stops heartbeating.

Behavioural rules you can rely on

- `heartbeat` every **15 minutes ± 45 seconds** of jitter while energized.
- On power loss, firmware ≥ 1.3 attempts a single `power_lost` message from capacitor reserve. It succeeds roughly **70%** of the time.
- On power return, the device sends `boot` and then `power_restored`, typically within 20 seconds.
- **At-least-once delivery**. Duplicates happen. Retries happen for up to 6 hours from a device that was offline, so you can receive a very stale `power_lost` long after the event.
- **~4% of the fleet is offline at any moment** for unrelated reasons — dead modem, vandalism, water ingress, expired SIM.

What the device does *not* tell you

No current, no voltage magnitude, no direction of flow, no phase, no impedance, no fault type. Just live or not live. Do not design around data you wish you had.

3. The pole registry

A one-time export, CSV. This is the department's asset database.

```
pole_id,lat,lon,feeder_id,dt_id,seq_on_line,parent_pole_id,pole_type,ward,pincode,device_id
P-024431,12.968214,77.594612,F-07-03,D-0112,14,P-024430,LT-9m-PCC,W-084,560078,KSPDB-SD07-D0112-4431
P-024432,12.968901,77.594330,F-07-03,D-0112,15,P-024431,LT-9m-PCC,W-084,560078,KSPDB-SD07-D0112-4432
P-024433,12.969455,77.593980,F-07-03,D-0112,,,LT-8m-Steel,W-084,560078,
```

Column	Notes
<code>pole_id</code>	Primary key.
<code>lat, lon</code>	Surveyed GPS. Accurate to about ±4 m. Always present, always trustworthy .
<code>feeder_id</code>	Which 11 kV feeder ultimately supplies this pole. Always present.
<code>dt_id</code>	Which distribution transformer supplies this pole. Always present.
<code>seq_on_line</code>	Position along the LT line from the transformer, 1 = closest. Missing for about 60% of DTs (see below).
<code>parent_pole_id</code>	The pole immediately upstream. Missing wherever <code>seq_on_line</code> is missing .
<code>pole_type</code>	Material and height. Cosmetic for this exercise.
<code>ward, pincode</code>	Administrative. <code>pincode</code> is missing for ~3% of rows.

Column	Notes
device_id	Empty where no device is fitted ($\approx 9\%$ of poles).

Separately you get the transformer registry:

```
dt_id,feeder_id,lat,lon,capacity_kva,households_served
D-0112,F-07-03,12.967801,77.595120,250,318
```

The missing-topology problem

For roughly 60% of distribution transformers, `seq_on_line` and `parent_pole_id` are empty. Those DTs were commissioned before the asset digitization drive, and nobody recorded the order of poles along the line. You know which DT each pole belongs to, and you know exactly where each pole is, but not which pole feeds which.

This is not a bug in the data export and we will not be sending you a corrected file. It is the state of the world.

Consider it the central design question of the assignment. Some directions candidates have taken, none of which is the official answer:

- Infer the line order geometrically from pole coordinates and the transformer location, and say honestly how often that inference will be wrong.
- Fall back to a coarser localization (DT-level rather than span-level) where topology is unknown, and be explicit in the UI about which kind of answer the operator is looking at.
- Use observed outage history to learn the topology over time — poles that go dark together are probably adjacent.
- Push the problem back to the department: specify the survey you would ask for, and what it would cost them.

Whatever you choose, we want the reasoning, the failure modes, and a clear statement of what the system does in the 60% case versus the 40% case.

4. The scheduled outage feed

The department publishes planned load shedding and maintenance shutdowns. Assume this API exists and mock it:

```
GET /scheduled-outages?from=2026-07-29T00:00:00Z&to=2026-07-30T00:00:00Z
```

```
[
  {
    "id": "SO-2026-07-29-014",
    "scope": "feeder",
    "target_id": "F-07-03",
    "start": "2026-07-29T10:00:00Z",
    "end": "2026-07-29T12:30:00Z",
    "reason": "Planned maintenance - jumper replacement"
  },
  {
    "id": "SO-2026-07-29-021",
    "scope": "dt",
    "target_id": "D-0112",
    "start": "2026-07-29T14:00:00Z",
    "end": "2026-07-29T15:00:00Z",
    "reason": "Load shedding"
  }
]
```

Real-world caveats that apply: shutdowns start late and overrun by 20–40 minutes routinely, and about one in ten is cancelled without the feed being updated. Treating this feed as gospel will cause you to miss real faults during a window where nothing was actually switched off.

5. Geocoding and PIN codes

You need to output a PIN code with each fault, and `pincode` is missing for ~3% of poles.

You may use any offline dataset or public API you like. If you use a hosted geocoding service, the deployed public URL must still work for us without our own API key — so either commit a bounded offline dataset, ship your key via environment variables you control, or degrade gracefully with a visible note in the UI. A submission that shows "geocoding unavailable" everywhere because the reviewer has no key counts as broken.

6. The simulator you must build

You are not being given a data generator. Building one is part of the work, because how you choose to simulate reveals whether you understood the physics.

At minimum, your simulator must let us:

1. Load the pole and transformer registries (generate synthetic ones matching the schemas and scale above — you do not need all 38,400 poles, but the shape must be realistic and at least a few thousand poles).
2. Inject a fault of each type: span fault, DT fault, feeder fault.
3. Produce the telemetry that such a fault would actually cause — including the 30% of dying messages that never arrive, and the firmware-1.2 devices that just go quiet.
4. Inject noise independently: a device dying while power is fine, a scheduled outage, out-of-order and duplicate messages.

5. Repair a fault, and produce the restoration telemetry.

Make it drivable from the UI or a single documented command. We will be using it as our primary way of evaluating whether your system works, so if it is awkward to operate, that costs you directly.

7. Performance targets

State whether you meet these, and measure rather than guess.

Metric	Target
Fault occurrence → localized ticket visible in UI	< 120 s (p95)
Ingest throughput sustained	≥ 500 msg/s
Ingest burst tolerated without data loss	5,000 messages in 10 s
Operator console load, incident list	< 2 s
Restoration → ticket auto-verified	< 120 s

You will not be penalised for missing a target you have measured, documented, and explained. You will be penalised for claiming one you never tested.

Next: [03-deliverables-and-submission.md](#) (03-deliverables-and-submission.md).